Algoritmusok

Oszd meg és uralkodj

1. Osztuk fel a problémát több kisebb részre
2. Rekurzívan futtassuk az algoritmust a kisebb részeket
3. A megoldást helyezzük össze

Összefésüléses rendezés: $\Theta(n \log n)$

Gyors rendezés: $\Theta(n \log n)$

Bináris keresés: $\Theta(\log n)$

Összefésüléses rendezés

```
MergeSort(A, p, r)
    q = (p+r+1)/2

    MergeSort(A, p, q)
    MergeSort(A, q+1, r)
    Merge(A, p, q, r)
```

```
Merge(A, p, q, r)
    i=0, j=0
    for i=p to q
        B[i] = A[i]
        i=i+1
    for j=q+1 to r
        C[j] = A[j]
        j=j+1
    i=0, j=0
    for x=p to r
        if B[i] < C[i] then
            A[x] = B[i]
            i=i+1
        else
            A[x] = C[j]
            j=j+1
```

Bináris keresés

```
BinarySearch(A, p, r, x)
    q = p+r+1 /2
    ~~if A[q~~
    if p = r then
        ~~return~~
        if A[p] = x then
            return p
        else return nil
    if A[q] ≤ x then
        return BinarySearch(A, p, q, x)
    else
        return BinarySearch(A, q+1, r, x)
```

Gyors rendezés

```
QuickSort(A, p, r)
    pivot = r
    if p ≤ r ~~then return~~ then
    i := Partition(A, p, r-1, pivot)
    ~~q = p+r+1/2~~
    QuickSort(A, p, i)
    QuickSort(A, i+1, r)
```

```
QuickSort(A, p, r)
    if p < r then
        pivot := r
        i := Partition(p, p, r-1, pivot)
        QuickSort(A, p, i)
        QuickSort(A, i+1, r)
```

```
Partition(A, p, r, pivot)
    i := p
    for j := p to r do
        if A[j] < A[pivot] then
            csere(A, i, j)
            i := i+1
    csere(A, i, pivot)
    return i
```